

FraudGuard — Documentación Backend v2

FraudGuard — Documentación Técnica Backend v2

Universidad de Antioquia · Estructura de Datos 2026-1 Proyecto: Sistema de Detección de Fraudes en Transacciones Financieras

Tabla de contenidos

1. [Descripción general](#)
 2. [Arquitectura y tecnologías](#)
 3. [Estructuras de datos](#)
 4. [Persistencia en disco](#)
 5. [Cálculo de dimensiones](#)
 6. [Detección de fraude \(KNN\)](#)
 7. [API REST — referencia completa](#)
 - 7.1 [Convención de nombres JSON](#)
 - 7.2 [Manejo de errores](#)
 - 7.3 [Transacciones](#)
 - 7.4 [Alertas](#)
 - 7.5 [Detección](#)
 - 7.6 [Sistema](#)
 8. [Modelos de datos \(DTOs\)](#)
 9. [Enumeraciones](#)
 10. [Cómo ejecutar](#)
-

1. Descripción general

FraudGuard es un backend REST construido con **Spring Boot** que detecta transacciones fraudulentas en tiempo real usando el algoritmo **KNN (K Nearest Neighbors)** sobre un **KD-Tree 5-dimensional**. Cada transacción se convierte en un punto en $\mathbb{R}^5$ y se clasifica según la densidad de fraudes confirmados en su vecindad.

Flujo principal

POST /transacciones

- Validar campos obligatorios
 - Calcular 5 dimensiones (D1-D5)
 - KNN sobre el KD-Tree actual → estado_alerta automático
 - Persistir 128 bytes en transacciones.dat
 - Insertar en Hash Table (índice O(1) por ID)
 - Insertar nodo en KD-Tree → responder 201 con TransaccionCompletaDTO
-

2. Arquitectura y tecnologías

Componente	Tecnología
Framework	Spring Boot 3.x
Lenguaje	Java 21
Build	Maven (mvnw)
Serialización JSON	Jackson con estrategia global SNAKE_CASE
Persistencia	Archivo binario de registros de longitud fija (128 bytes)
Índice primario	Hash Table con encadenamiento separado
Índice espacial	KD-Tree 5D (implementación propia)
Logging	SLF4J + Logback (via Lombok @Slf4j)
Puerto	8080

Configuración clave (application.properties)

```
spring.jackson.property-naming-strategy=SNAKE_CASE
```

Esta línea hace que **todos** los campos Java en camelCase se serialicen y deserialicen automáticamente como snake_case. Por ejemplo, idTransaccion → id_transaccion, estadoAlerta → estado_alerta.

3. Estructuras de datos

3.1 KD-Tree 5D

Árbol binario de búsqueda espacial donde cada nivel discrimina por una dimensión diferente (ciclando D1 → D2 → D3 → D4 → D5 → D1 → ...).

Operaciones:

Operación	Complejidad promedio	Complejidad peor caso
Insertar	$O(\log n)$	$O(n)$
KNN (k vecinos)	$O(k \cdot \log n)$	$O(k \cdot n)$
Búsqueda por rango (hipercubo 5D)	$O(\sqrt{n} + m)$	$O(n)$
Marcar eliminado	$O(\log n)$	$O(n)$

Eliminación lógica en el árbol: los nodos se marcan como eliminados (marcarEliminado(id)) sin reestructurar el árbol. Al actualizar una transacción se elimina el nodo viejo y se inserta uno nuevo con las nuevas dimensiones.

Método getProfundidadMax(): recorre el árbol recursivamente y retorna la profundidad máxima alcanzada. Se usa en las estadísticas del sistema.

3.2 Hash Table

Tabla hash con **encadenamiento separado** (arreglo de listas enlazadas). Mapea idTransaccion (String) → numRegistro (int).

Operaciones:

Operación	Complejidad
<code>insert(id, numReg)</code>	$O(1)$ amortizado
<code>search(id)</code>	$O(1)$ promedio
<code>delete(id)</code>	$O(1)$ promedio
<code>contains(id)</code>	$O(1)$ promedio

Cuando `search(id)` no encuentra el ID retorna `-1`.

4. Persistencia en disco

El archivo `transacciones.dat` almacena registros de **128 bytes fijos**. La posición de un registro en el archivo es `numRegistro * 128`.

Layout del registro (128 bytes)

Offset	Bytes	Campo	Tipo
0	20	<code>id_transaccion</code>	UTF-8, relleno con <code>\0</code>
20	20	<code>num_cuenta</code>	UTF-8, relleno con <code>\0</code>
40	8	<code>monto</code>	double big-endian
48	8	<code>timestamp</code>	long big-endian (Unix epoch en segundos)
56	8	<code>d1_monto_norm</code>	double
64	8	<code>d2_hora</code>	double
72	8	<code>d3_frecuencia</code>	double
80	8	<code>d4_tipo</code>	double
88	8	<code>d5_desviacion</code>	double
96	1	<code>estado_alerta</code>	byte (0-4)
97	1	<code>deleted</code>	byte (0=activo, 1=eliminado)
98	30	<code>padding</code>	ceros

Eliminación lógica: se sobrescribe solo el byte en offset 97 con 1. El registro permanece en el archivo pero se ignora en todas las consultas.

Actualización de estado_alerta: se sobrescribe solo el byte en offset 96, sin releer ni reescribir el registro completo.

5. Cálculo de dimensiones

Cada transacción se convierte en un punto de 5 dimensiones antes de insertarse en el KD-Tree.

Dimensión	Campo	Cálculo	Rango
D1	<code>d1_monto_norm</code>	$(\text{monto} - \text{montoMin}) / (\text{montoMax} - \text{montoMin})$	$[0.0, 1.0]$
D2	<code>d2_hora</code>	$\text{hora} + \text{minuto} / 60.0$ del timestamp UTC	$[0.0, 23.99]$
D3	<code>d3_frecuencia</code>	Número de transacciones activas de la misma cuenta	$[0, \infty)$

Dimensión	Campo	Cálculo	Rango
		en las 24h previas	
D4	d4_tipo	RETIRO=0.0, DEPOSITO=0.5, TRANSFERENCIA=1.0	{0.0, 0.5, 1.0}
D5	d5_desviacion	Desviaciones estándar del monto respecto al promedio histórico de la cuenta	$(-\infty, \infty)$

Importante: D1 y D5 requieren estadísticas globales/por cuenta que se mantienen en memoria (DimensionCalculator). Si no hay historial previo, D1 retorna 0.5 y D5 retorna 0.0.

6. Detección de fraude (KNN)

Regla de clasificación (K = 5)

Vecinos confirmados como fraude Estado asignado

≥ 3	ALTA (2)
≥ 1	MEDIA (1)
0	NORMAL (0)

Solo se cuentan vecinos con estado_alerta = CONFIRMADO_FRAUDE (3).

Cuándo se ejecuta KNN

1. **Al crear** una transacción (POST /transacciones): KNN se ejecuta **antes** de insertar el registro en el árbol para evitar que el nuevo nodo aparezca como su propio vecino.
2. **Al actualizar** (PUT /transacciones/{id}): el nodo viejo se marca eliminado, se inserta el nuevo, y luego se reclasifica.
3. **Análisis explícito** (POST /deteccion/analizar/{id}): fuerza una reclasificación del registro existente en el árbol.
4. **Escaneo masivo** (POST /deteccion/escaneo-masivo): reclasifica todas las transacciones NORMAL y MEDIA.

Regla de no sobrescritura

Los estados CONFIRMADO_FRAUDE y FALSO_POSITIVO son decididos por un analista humano y **nunca se sobrescriben** por KNN automático.

7. API REST — referencia completa

Base URL: http://localhost:8080

7.1 Convención de nombres JSON

Todos los campos se serializan y deserializan en **snake_case**:

- idTransaccion ↔ id_transaccion

- estadoAlerta ↔ estado_alerta
- numCuenta ↔ num_cuenta
- d1MontoNorm ↔ d1_monto_norm
- etc.

7.2 Manejo de errores

Todas las respuestas de error tienen la misma estructura:

```
{
  "codigo": "NOT_FOUND",
  "mensaje": "Transacción no encontrada: TXN-ABC-001"
}
```

HTTP		codigo	Causa
400	BAD_REQUEST		Campo faltante, valor inválido o restricción de negocio
404	NOT_FOUND		ID no existe o registro eliminado lógicamente
409	CONFLICT		ID de transacción duplicado
422	UNPROCESSABLE		No hay suficientes registros en el árbol para KNN
500	IO_ERROR		Error de lectura/escritura en el archivo de datos
500	INTERNAL_ERROR		Error inesperado del servidor

7.3 Transacciones

POST /api/v1/transacciones — Crear transacción

Calcula las 5 dimensiones, ejecuta KNN para determinar el estado de alerta automáticamente, persiste en disco e indexa en memoria.

Request body:

```
{
  "id_transaccion": "TXN-2024-001",
  "num_cuenta": "ACC-123456",
  "monto": 1500.00,
  "timestamp": 1700000000,
  "tipo": "TRANSFERENCIA"
}
```

Campo		Tipo	Restricciones
id_transaccion		string	Obligatorio, máximo 19 caracteres, único
num_cuenta		string	Obligatorio, máximo 19 caracteres
monto		number	Obligatorio, > 0
timestamp		integer	Obligatorio, > 0 (Unix epoch en segundos)
tipo		string	Obligatorio: "RETIRO", "DEPOSITO" o "TRANSFERENCIA"

Respuestas:

- 201 Created → TransaccionCompletaDTO
- 400 Bad Request → validación fallida
- 409 Conflict → ID duplicado

GET /api/v1/transacciones — Listar con filtros y paginación

Query params:

Parámetro	Tipo	Default	Descripción
num_cuenta	string	—	Filtra por número de cuenta (exacto)
tipo	string	—	RETIRO, DEPOSITO o TRANSFERENCIA
estado_alerta	stringlint	—	Nombre del estado (NORMAL, ALTA, ...) o código numérico (0-4)
pagina	int	0	Página base-0
tamano	int	20	Elementos por página (1-100)

Respuesta 200 OK:

```
{
  "transacciones": [ /* array de TransaccionCompletaDTO */,
  "total_elementos": 42,
  "pagina_actual": 0,
  "total_paginas": 3
}
```

GET /api/v1/transacciones/{id} — Buscar por ID

Búsqueda O(1) a través de la Hash Table.

Respuestas:

- 200 OK → TransaccionCompletaDTO
- 404 Not Found → ID no existe o eliminado lógicamente

PUT /api/v1/transacciones/{id} — Actualizar transacción

Recalcula todas las dimensiones con los nuevos valores, reemplaza el nodo en el KD-Tree (marca eliminado e inserta nuevo) y reclasifica con KNN.

Request body:

```
{
  "monto": 2000.00,
  "tipo": "DEPOSITO",
  "timestamp": 1700003600
}
```

Campo	Tipo	Restricciones
monto	number	Obligatorio, > 0
tipo	string	Obligatorio: RETIRO, DEPOSITO o TRANSFERENCIA

Campo	Tipo	Restricciones
timestamp	integer null	Opcional — si es null mantiene el timestamp original

Respuestas:

- 200 OK → TransaccionCompletaDTO con dimensiones y estado recalculados
- 400 Bad Request
- 404 Not Found

DELETE /api/v1/transacciones/{id} — Eliminar (lógico)

Sobreescribe el byte deleted en el archivo sin borrar el registro. Lo elimina de la Hash Table y lo marca en el KD-Tree.

Respuestas:

- 204 No Content — eliminación exitosa
- 404 Not Found

7.4 Alertas**GET /api/v1/alertas — Listar alertas activas**

Retorna transacciones con estado MEDIA o ALTA. El parámetro nivel_minimo permite filtrar por severidad mínima (1=MEDIA, 2=ALTA).

Query params:

Parámetro	Tipo	Default	Descripción
nivel_minimo	int	1	Nivel mínimo de alerta (1=MEDIA, 2=solo ALTA)

Respuesta 200 OK:

```
{
  "total_alertas": 8,
  "alertas_alta": 3,
  "alertas_media": 5,
  "transacciones": [ /* array de TransaccionCompletaDTO */ ]
}
```

PATCH /api/v1/alertas/{id}/confirmar — Confirmar fraude

Cambia el estado a CONFIRMADO_FRAUDE (3). Este estado nunca es sobrescrito por KNN automático. No se puede confirmar una transacción ya confirmada.

Respuestas:

- 200 OK → TransaccionCompletaDTO
- 400 Bad Request → ya estaba confirmada

- 404 Not Found

PATCH /api/v1/alertas/{id}/descartar — Descartar (falso positivo)

Cambia el estado a FALSO_POSITIVO (4). No se puede descartar una transacción ya descartada.

Respuestas:

- 200 OK → TransaccionCompletaDTO
- 400 Bad Request → ya estaba descartada
- 404 Not Found

7.5 Detección

POST /api/v1/deteccion/analizar/{id} — Re-analizar con KNN

Ejecuta KNN sobre el registro existente en el árbol. Si el estado calculado difiere del actual lo actualiza en disco y en memoria. No sobrescribe CONFIRMADO_FRAUDE ni FALSO_POSITIVO.

Query params:

Parámetro	Tipo	Default	Descripción
k	int	5	Número de vecinos a considerar

Respuesta 200 OK:

```
{
  "id_transaccion": "TXN-2024-001",
  "k_utilizado": 5,
  "vecinos_fraude": 2,
  "estado_alerta_asignado": 1,
  "estado_alerta_nombre": "MEDIA"
}
```

Campo	Descripción
k_utilizado	K efectivo usado (puede ser menor al solicitado si hay pocos nodos)
vecinos_fraude	Cuántos de los k vecinos son CONFIRMADO_FRAUDE
estado_alerta_asignado	Código numérico del nuevo estado (0-4)
estado_alerta_nombre	Nombre del estado

GET /api/v1/deteccion/vecinos/{id} — Obtener K vecinos más cercanos

Retorna los K vecinos en el KD-Tree más cercanos a la transacción consultada, ordenados por distancia euclidiana creciente.

Query params:

Parámetro	Tipo	Default	Restricciones
k	int	5	1 ≤ k ≤ 50

Respuesta 200 OK:

```
{
  "id_transaccion_consulta": "TXN-2024-001",
  "k": 5,
  "vecinos": [
    {
      "posicion": 1,
      "id_transaccion": "TXN-2024-005",
      "monto": 1480.00,
      "tipo": "TRANSFERENCIA",
      "estado_alerta": 3,
      "estado_alerta_nombre": "CONFIRMADO_FRAUDE",
      "distancia_euclidiana": 0.0342
    }
  ]
}
```

Respuestas:

- 200 OK
- 400 Bad Request → k fuera del rango 1-50 o insuficientes registros en el árbol
- 404 Not Found

POST /api/v1/deteccion/rango — Búsqueda por hipercubo 5D

Encuentra todas las transacciones cuyas 5 dimensiones caen dentro del rango especificado. Los campos son opcionales; los ausentes o nulos se interpretan como $-\infty$ (para mínimos) o $+\infty$ (para máximos).

Request body:

```
{
  "d1_monto_min": 0.2,
  "d1_monto_max": 0.8,
  "d2_hora_min": 22.0,
  "d2_hora_max": 23.99,
  "d3_frecuencia_min": null,
  "d3_frecuencia_max": null,
  "d4_tipo_min": 0.9,
  "d4_tipo_max": 1.0,
  "d5_desviacion_min": null,
  "d5_desviacion_max": null
}
```

Campo		Dimensión	Descripción
d1_monto_min/max	D1		Monto normalizado [0.0-1.0]
d2_hora_min/max	D2		Hora del día [0.0-23.99]
d3_frecuencia_min/max	D3		Transacciones de la cuenta en últimas 24h
d4_tipo_min/max	D4		Tipo codificado: 0.0=RETIRO, 0.5=DEPOSITO, 1.0=TRANSFERENCIA
d5_desviacion_min/max	D5		Desviaciones estándar del monto

Respuesta 200 OK:

```
{
  "total": 3,
  "transacciones": [ /* array de TransaccionCompletaDTO */ ]
}
```

POST /api/v1/deteccion/escaneo-masivo — Reclasificar en lote

Re-ejecuta KNN sobre todas las transacciones activas con estado NORMAL o MEDIA. Útil después de confirmar fraudes para actualizar el resto del dataset. No afecta CONFIRMADO_FRAUDE ni FALSO_POSITIVO.

Respuesta 200 OK:

```
{
  "mensaje": "Escaneo masivo completado",
  "transacciones_analizadas": 35,
  "alertas_nuevas_alta": 4,
  "actualizados": 7
}
```

7.6 Sistema

GET /api/v1/sistema/estadisticas — Estadísticas completas

Respuesta 200 OK:

```
{
  "total_transacciones": 42,
  "total_eliminadas": 3,
  "distribucion_alertas": {
    "normal": 28,
    "media": 8,
    "alta": 3,
    "confirmado_fraude": 2,
    "falso_positivo": 1
  },
  "hash_table": {
    "buckets_totales": 101,
    "factor_carga": 0.42,
    "colisiones": 5
  },
  "kdtree": {
    "total_nodos": 42,
    "profundidad_max": 7
  }
}
```

Campo	Descripción
total_transacciones	Registros activos (deleted = 0)
total_eliminadas	Registros con eliminación lógica (deleted = 1)
distribucion_alertas	Conteo por cada estado de alerta (solo activos)
hash_table.factor_carga	size / buckets_totales
hash_table.colisiones	Entradas que comparten bucket
kdtree.profundidad_max	Profundidad máxima del árbol (calculada recursivamente)

GET /api/v1/sistema/salud — Estado del sistema

Respuesta 200 OK:

```
{
  "estado": "OK",
  "archivo_datos": "OK",
  "hash_table": "OK (42 entradas)",
  "kd_tree": "OK (42 nodos activos)",
  "registros_totales": 45,
  "hash_table_carga": 0.42,
  "kdtree_nodos": 42,
  "archivo_bytes": 5760
}
```

Campo	Descripción
estado	"OK" o "DEGRADADO" según si el archivo es accesible
hash_table_carga	Factor de carga numérico (0.0-1.0)
kdtree_nodos	Nodos activos en el árbol
archivo_bytes	Tamaño del archivo transacciones.dat en bytes (registros_totales * 128)

POST /api/v1/sistema/seed — Cargar datos de ejemplo

Inserta ~30 transacciones de ejemplo con distintos patrones de fraude.

Query params:

Parámetro	Tipo	Default	Descripción
forzar	boolean	false	Si es true, reinicia todo antes de sembrar; si es false falla si el archivo ya tiene datos

Respuesta 200 OK:

```
{
  "mensaje": "Datos de ejemplo cargados correctamente",
  "registrosCargados": 30,
  "hashEntradas": 30,
  "kdTreeNodos": 30
}
```

Respuestas:

- 200 OK
- 400 Bad Request → archivo con datos y forzar=false

8. Modelos de datos (DTOs)

TransaccionCompletaDTO

Respuesta estándar para operaciones sobre una transacción.

```
{
  "offset_disco":      0,
  "id_transaccion":    "TXN-2024-001",
  "num_cuenta":        "ACC-123456",
  "monto":             1500.00,
  "timestamp":         1700000000,
  "tipo":              "TRANSFERENCIA",
  "estado_alerta":     1,
  "estado_alerta_nombre": "MEDIA",
  "deleted":           false,
  "dimensiones": {
    "d1_monto_norm":   0.432,
    "d2_hora":         14.5,
    "d3_frecuencia":   2.0,
    "d4_tipo":         1.0,
    "d5_desviacion":   0.87
  }
}
```

Campo	Tipo	Descripción
offset_disco	int	Posición en bytes en el archivo (numRegistro × 128)
estado_alerta	int	Código numérico (0-4)
estado_alerta_nombre	string	Nombre del enum
deleted	boolean	true si fue eliminado lógicamente
dimensiones	object	Las 5 dimensiones calculadas para el KD-Tree

PaginaDTO (respuesta paginada)

```
{
  "transacciones":    [ /* array */ ],
  "total_elementos":  42,
  "pagina_actual":    0,
  "total_paginas":    3
}
```

AlertaListaDTO

```
{
  "total_alertas": 8,
  "alertas_alta": 3,
  "alertas_media": 5,
  "transacciones": [ /* array de TransaccionCompletaDTO */ ]
}
```

AnalisisResultadoDTO

```
{
  "id_transaccion":    "TXN-2024-001",
  "k_utilizado":       5,
  "vecinos_fraude":    2,
  "estado_alerta_asignado": 1,
}
```

```
"estado_alerta_nombre": "MEDIA"
}
```

VecinosResultadoDTO

```
{
  "id_transaccion_consulta": "TXN-2024-001",
  "k": 5,
  "vecinos": [
    {
      "posicion": 1,
      "id_transaccion": "TXN-2024-005",
      "monto": 1480.00,
      "tipo": "TRANSFERENCIA",
      "estado_alerta": 3,
      "estado_alerta_nombre": "CONFIRMADO_FRAUDE",
      "distancia_euclidiana": 0.0342
    }
  ]
}
```

RangoBusquedaDTO (body del POST /deteccion/rango)

```
{
  "d1_monto_min": 0.2,
  "d1_monto_max": 0.8,
  "d2_hora_min": null,
  "d2_hora_max": null,
  "d3_frecuencia_min": null,
  "d3_frecuencia_max": null,
  "d4_tipo_min": 0.9,
  "d4_tipo_max": 1.0,
  "d5_desviacion_min": null,
  "d5_desviacion_max": null
}
```

Los campos null se interpretan como $-\infty$ (mínimos) o $+\infty$ (máximos), logrando búsquedas sin límite en esa dimensión.

RangoResultadoDTO

```
{
  "total": 3,
  "transacciones": [ /* array de TransaccionCompletaDTO */ ]
}
```

9. Enumeraciones

EstadoAlerta

Código	Nombre	Descripción
0	NORMAL	Sin señales de fraude
1	MEDIA	1-2 vecinos confirmados como fraude
2	ALTA	≥ 3 vecinos confirmados como fraude
3	CONFIRMADO_FRAUDE	Confirmado manualmente por analista — no sobreescribible
4	FALSO_POSITIVO	Descartado por analista — no sobreescribible

TipoTransaccion

Nombre	Valor D4	JSON
RETIRO	0.0	"RETIRO"
DEPOSITO	0.5	"DEPOSITO"
TRANSFERENCIA	1.0	"TRANSFERENCIA"

10. Cómo ejecutar

Requisitos

- Java 21 (Temurin o Eclipse Adoptium recomendado)
- Maven (incluido via mvnw)

Compilar y ejecutar desde código fuente

```
cd backend
```

```
# Compilar (requiere Java 21 en JAVA_HOME)
export JAVA_HOME=/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home
./mvnw package -DskipTests
```

```
# Ejecutar
/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java \
  -jar target/fraud-detector-*.jar
```

Ejecutar el JAR directamente

```
/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java \
  -jar backend/executable/fraud-detector.jar
```

El servidor arranca en `http://localhost:8080`. El archivo de datos `transacciones.dat` se crea en el directorio de trabajo.

Cargar datos de ejemplo (opcional)

```
curl -X POST "http://localhost:8080/api/v1/sistema/seed?forzar=true"
```

Verificar que funciona

```
curl http://localhost:8080/api/v1/sistema/salud
```

Respuesta esperada:

```
{ "estado": "OK", ... }
```

Documentación generada para la versión 2 del backend — Mayo 2026